Cross-Study Empirical Reproduction and Evaluation of Token-based Software Vulnerability Detection Systems

SEMIU AKANMU

Department of Computer Science, North Dakota State University, Fargo, North Dakota, United States.

semiu.akanmu@ndsu.edu

SIMONE LUDWIG

Department of Computer Science, North Dakota State University, Fargo, North Dakota, United States.

simone.ludwig@ndsu.edu

Advances in machine learning for software vulnerability detection have introduced a variety of modeling approaches, including classical token-based methods, deep neural architectures, and hybrid combinations. However, these approaches are typically evaluated under differing datasets, preprocessing pipelines, and performance metrics, limiting the comparability and reproducibility of reported results. This paper systematically reproduces and comparatively evaluates vulnerability detection models from prior studies under a unified experimental setting. We reproduce classical bag-of-words (BoW) models, convolutional neural networks (CNNs), and hybrid architectures combining neural representations with tree-based classifiers, evaluating all models on a common dataset using area under the precision–recall curve (AUPRC) as the primary metric, alongside precision, recall, and F1 scores. In addition to point estimates, we employ bootstrap resampling to compute confidence intervals and conduct pairwise statistical comparisons between models. Our results show that reproduction fidelity is limited in absolute terms but preserved in relative performance trends. Neural models consistently outperform classical approaches, achieving substantial improvements in AUPRC (up to 0.75 compared to 0.18–0.33 for BoW models), with statistically significant differences ($p < 0.001$). Hybrid models exhibit context-dependent behavior, while a CNN combined with a random forest significantly improves performance over its standalone counterpart ($p = 0.024$), a CNN combined with extra trees does not yield a statistically significant improvement. Furthermore, differences among top-performing neural models are not statistically significant, indicating comparable effectiveness. These findings highlight the importance of standardized evaluation protocols, appropriate metrics for imbalanced data, and statistical testing in assessing vulnerability detection systems.

CCS CONCEPTS • Software and its engineering • Security and privacy

Additional Keywords and Phrases: software vulnerability detection, empirical evaluation, token-based representation learning, machine learning

1 INTRODUCTION

Software vulnerabilities remain a persistent and critical challenge in modern software systems [1], [2]. Despite advances in static analysis, dynamic testing, and secure development practices, thousands of new

vulnerabilities are disclosed annually, many stemming from subtle coding errors that evade manual review and rule-based detection tools [3], [4]. However, the increasing availability of large open-source repositories [5], [6] has enabled data-driven approaches to vulnerability detection, particularly machine learning (ML) techniques that treat source code as structured text and learn predictive patterns directly from tokenized representations [7], [8], [9].

Over the past decade, multiple token-based vulnerability detection systems have been proposed, ranging from classical bag-of-words models [9], [10], [11], [12] to deep neural architectures incorporating convolutional and recurrent networks [13], [14], [15], [16]. For example, Machine Learning Methods for Software Vulnerability Detection [17] explored count-based lexical approaches using Naïve Bayes and tree ensembles, while Automated Software Vulnerability Detection with machine learning [16] investigated hybrid deep-feature and ensemble methods. Also, deep representation learning, combined with ensemble classifiers, demonstrated strong precision–recall performance on large curated datasets in Automated Vulnerability Detection in Source Code Using Deep Representation Learning [18]. Although these studies report promising results, meaningful cross-study comparison remains difficult. Each work differs in dataset composition, labeling methodology, tokenization strategy, class imbalance, hyperparameter selection, and evaluation metrics. Consequently, it remains unclear whether reported performance improvements stem primarily from architectural advances or from differences in experimental environments. Class imbalance also remains a major methodological challenge across machine-learning-based security applications, and its effects can vary with the learning algorithm and the strategy used to address it. For example, Xie et al. [19] investigated random oversampling, the Synthetic Minority Over-sampling Technique (SMOTE), and a customized SMOTE strategy across several machine-learning classifiers for network intrusion detection. Their results showed that resampling effects differed across attack categories and classifiers, and improvements in minority-class detection did not necessarily translate into uniform gains across all evaluation measures.

This paper presents a cross-study empirical reproduction and evaluation of model-family-representative token-based vulnerability detection systems. We faithfully re-implemented selected classical, hybrid, and deep architectures as described in their original publications, preserving their respective tokenization pipelines, embedding strategies, batch configurations, and hyperparameters. We evaluated all methods on a shared dataset using identical train/validation/test splits, isolating dataset effects while retaining architectural fidelity. Our goal is not to propose a new detection model or a benchmark-performing model, but to examine how well previously reported approaches generalize beyond their original evaluation settings. By applying heterogeneous methods to a common dataset under consistent partitioning, we provide a systematic empirical assessment of their precision–recall performance. Therefore, to address the limitations of cross-study comparison in prior works, this study investigates the following research questions:

RQ1: How do representative token-based vulnerability detection systems perform when faithfully reproduced and evaluated under a shared dataset and identical data splits?

RQ2: Under identical dataset partitions, how do classical count-based models compare to deep and hybrid neural architectures in precision–recall performance?

RQ3: Does combining neural feature extraction with ensemble classifiers consistently improve vulnerability detection performance compared to standalone neural models?

In answering these research questions, the contributions of this work are:

1. Faithful cross-study reproduction of representative token-based vulnerability detection systems spanning count-based, hybrid, and deep architectures.
2. Precision–recall–focused analysis of model behavior under class imbalance, accompanied by statistical comparison.

By focusing on external reproducibility and comparative evaluation, this study contributes to the growing need for empirical and methodological transparency in machine learning–based vulnerability detection research. The remaining parts of this paper are organized as follows: Section 2 details the reproduced token-based detection systems. Section 3 describes the experimental design framework, while Section 4 reports the experimental results and interpretation. Section 5 discusses the findings, and Section 6 identifies potential threats to their validity. The conclusion is presented in Section 7.

2 REPRODUCED TOKEN-BASED DETECTION SYSTEMS

The six (6) reproduced systems are derived from three (3) studies: Chernis and Verma's Machine Learning Methods for Software Vulnerability Detection [17], Harer et al.'s Automated Software Vulnerability Detection with machine learning [16], and Russell et al.'s Automated Vulnerability Detection in Source Code Using Deep Representation Learning [18]. Unless otherwise stated, implementation details follow the specifications described in the respective original works.

2.1 Bag-of-Words + Naïve Bayes (Chernis and Verma)

This model reproduces a count-based lexical approach. After preprocessing that removes line formats, block comments, and string literals, we tokenized the source code at the word level and represented it as a unigram-type Bag-of-Words (BoW). No semantic embedding or contextual encoding was applied. A standard multinomial Naïve Bayes classifier was employed. It used the default smoothing parameters consistent with standard Naïve Bayes implementations. We imposed no feature selection beyond the BoW representation and applied no dimensionality reduction, especially since the original author reported performance degradation when they applied Principal Component Analysis (PCA). The classifier was trained on the training partition using maximum likelihood estimation. We used validation only for performance monitoring, not for architectural tuning, and performed no additional resampling. The classifier inherently estimates class priors from the training data, assuming conditional independence among tokens and that the vulnerability signal is reflected in the token frequency distribution.

2.2 Bag-of-Words + Extra Trees (Harer et al.)

This model extends the BoW representation with an ensemble tree classifier, inspired by the tree-based baseline. The representation treats source code as text. The preprocessing stage included removing line formatting and block comments. The lexical tokenization used a custom lexer consistent with the original study. Tokens included keywords, operators, identifiers, and literals, which the original study mapped to different bins before converting to a BoW vector representation. The classifier used is an Extremely Randomized Trees (Extra Trees) classifier, with hyperparameters such as the number of trees, the default tree depth, and Gini impurity as the splitting criterion, consistent with the reported configurations.

2.3 CNN + Extra Trees (Harer et al.)

This hybrid architecture combines deep feature extraction with tree-based classification. The tokenization approach is like what is described in Section 2.2, except that the identifiers were abstracted into generic placeholders to reduce vocabulary size. This model used pretrained Word2Vec embeddings to initialize the CNN's embedding layer. The embeddings were fine-tuned during the supervised training. The architecture includes a convolutional layer for a TextCNN-style feature extraction stage, but without the final dense classification layer. The extracted convolutional features are passed to an Extra Trees classifier. The hyperparameters used, including convolution filter sizes, batch size, learning rate, the number of filters per size, ReLU activation, and Dropout, match the original paper.

2.4 Word2vec + Full CNN (Harer et al.)

This model's tokenization, embedding strategy, hyperparameter configuration, and cross-entropy loss training framework match Section 2.2. However, its TextCNN-style architecture includes a final dense classification layer. Therefore, the convolution layer's extracted features are fed into the classification head. No resampling technique is applied other than threshold tuning on the validation set during the precision-recall evaluation. Although this model also assumes local token co-occurrence patterns contain vulnerability signals, it prefers the performance of the neural network (NN) classification layer.

2.5 Full CNN (Russell et al.)

This model reproduces the standalone CNN classifier. The lexical tokenization used a custom lexer and lexical normalization, reducing the vocabulary to approximately 156 standardized tokens. The identifiers are abstracted, and the literals are normalized except for integers. The embedding strategy is a trainable 13-dimensional token embedding randomly initialized with no external pretraining. The model architecture's embedding layer, convolutional filters of size 9 and 512 channels, batch normalization, ReLU activation, global max pooling, fully connected layers (64 and 16 units), dropout of 0.5, and Softmax output are consistently reproduced. The hyperparameters, such as a sequence length constraint of 10–500 tokens and an Adam optimizer with a learning rate, are implemented as specified.

2.6 CNN + RF (Russell et al.)

This hybrid model uses the same tokenization approach, embedding strategy (learned 13-dimensional embeddings during supervised training), model architecture, and hyperparameter configuration as described in Section 2.5. The key difference is that the convolutional layer's extracted features are fed into a Random Forest (RF) classifier. The RF uses the default splitting criteria, and the tree count matches the reported configuration. The class imbalance is handled by weighted cross-entropy during CNN training, but the extracted features fed into the RF are not rebalanced. The model training framework assumes that an ensemble classifier enhances generalization over the Softmax layer and that a hybrid architecture mitigates neural overfitting.

2.7 Reproduction Fidelity

We reproduced each model as faithfully as possible by preserving the preprocessing pipeline, tokenization strategy, embedding representation, neural architecture, optimizer, learning rate, batch size, training epochs,

and other reported hyperparameters from the original studies. The only deliberate modification was the use of a common benchmark dataset and identical train/validation/test partitions across all models to enable controlled cross-study comparison. Consequently, differences between reproduced and originally reported results should primarily reflect dataset and evaluation protocol differences rather than architectural modifications. To facilitate independent verification, the complete source code, configuration files, preprocessing scripts, trained models, and reproduction instructions are publicly released[1]. The repository documents all implementation choices, including any required assumptions when original studies omitted implementation details.

3 EXPERIMENTAL DESIGN

3.1 Dataset and Partitioning

The dataset used in this study was originally curated by [15]. It is a collection of deduplicated C/C++ functions from six open-source projects.[2] In total, the dataset contains 480 vulnerable code files and 40191 non-vulnerable code files, where an individual file contains a function that serves as the unit of analysis. We prepared this dataset for token-based model building by converting it to a tabular format, where the code text column contains the text from each file, and the class column indicates whether the file is vulnerable. All models were evaluated on the same curated dataset of C/C++ functions. The dataset was partitioned into three sets: training, validation, and test set, using a 70:15:15 ratio. Partitions were stratified and fixed. The dataset exhibits class imbalance, reflecting realistic vulnerability prevalence.

3.2 Model Training Protocol

Each reproduced model was trained independently using its original training configuration, which includes tokenization and lexical abstraction (method-specific), embedding initialization strategy (random or pretrained as reported), network architecture and depth, optimizer choice and learning rate, batch size, sequence length constraints, dropout rates, and loss formulation (including any class-weighting originally described). No cross-model hyperparameter harmonization was performed. For models originally incorporating class-weighted loss functions (e.g., weighted cross-entropy), as seen in [18], those configurations were preserved. For classical machine learning methods without built-in weighting mechanisms, training was performed using default class priors derived from the dataset.

3.3 Evaluation Metrics

To address RQ1–RQ3 (stated in the Introduction; Section 1) under realistic imbalance conditions, the evaluation focused on precision–recall performance rather than accuracy. The following metrics were

[1] Source code of the reproduced and evaluated cross-study models: https://github.com/Semiu/cve-automated-detection/tree/main

[2] Original data can be downloaded from https://github.com/DanielLin1986/TransferRepresentationLearning/blob/master/Data/VulnerabilityData/6_projects_functions.zip

computed on the held-out test set: (a) Precision–Recall Curve, (b) Area Under the Precision–Recall Curve (AUPRC), and (c) F1-score at the validation-optimized threshold.

3.4 Comparative Analysis Framework

The comparative analysis framework, conducted in the Precision-Recall space, ensures that we measure each reproduced system's performance under identical dataset partitions. Reported AUPRC and threshold-based metrics were compared against claims from the original studies to assess relative generalization behavior. The architecture comparison also guides the grouping of the models into three categories: namely, (a) count-based (for example, BoW + Naïve Bayes, BoW + Extra Trees), (b) standalone neural (for example, Full CNN), and (c) hybrid neural-ensemble (CNN + RF, CNN + Extra Trees). The hybridization effect was studied across architectures derived from the same neural feature extractor: Full CNN vs. CNN + RF. Differences in AUPRC and F1 were evaluated to determine whether ensemble classification consistently improves generalization.

3.5 Statistical Testing

This study employs (a) bootstrapped confidence intervals for AUPRC, (b) pairwise comparisons between models using stratified bootstrap resampling of the test set, and (c) statistical significance assessment at $\alpha = 0.05$, to determine whether observed performance differences are statistically significant. This procedure ensures that conclusions are supported by inferential analysis rather than descriptive comparison alone.

3.6 Implementation Environment

All models were implemented in Python, using applicable machine learning and deep learning libraries such as PyTorch and Scikit-Learn. The implementations were run on consistent hardware (1 node, 12 CPU cores, 64GB RAM, and 500GB hard disk) and runtime environments to eliminate computational variability as a confounding factor. Random seeds were fixed where applicable to ensure reproducibility.

4 EXPERIMENTAL RESULTS AND INTERPRETATION

This section presents the results and empirical findings used to answer the research questions RQ1–RQ3. We evaluated each of the reproduced systems using Area Under the Precision-Recall Curve (AUPRC), F1, Precision, Recall, and 95% Confidence Interval (CI), calculated from a bootstrapped AUPRC. Table 1 presents each reproduced model's performance on the test set.

Table 1: Performance of Reproduced Models on the Test Set (AUPRC, 95% Confidence Intervals and Threshold-Based Metrics)

Model	AUPRC	95% CI (AUPRC)	Precision@0.5	Recall@0.5	Best F1	Threshold
Chernis BoW + NB	0.3279	[0.2265, 0.4453]	0.4800	0.4068	0.4483	0.90
Harer's BoW + NB	0.1748	[0.1026, 0.2875]	0.5455	0.0833	0.2158	0.20
Harer's word2vec + CNN	0.7550	[0.6593, 0.8415]	0.9074	0.6901	0.7840	0.17
Harer's CNN + ET	0.6690	[0.5403, 0.7829]	0.9167	0.4648	0.5934	0.14
Russell's CNN	0.6553	[0.5388, 0.7666]	0.6974	0.7465	0.7211	0.77
Russell's CNN + RF	0.7208	[0.6220, 0.8191]	0.8545	0.6620	0.7460	0.46

Chernis' BoW + NB model achieved moderate ranking performance (AUPRC = 0.3279) with balanced precision (0.4800) and recall (0.4068). The relatively high optimal threshold (0.90) indicates that strong confidence was required to achieve the best precision–recall balance. Harer's BoW + ET's model produced the weakest overall ranking performance (AUPRC = 0.1748).

Although precision remained relatively high (0.5455), recall was very low (0.0833), indicating that the model identified only a small proportion of vulnerable functions despite making comparatively accurate positive predictions. Harer's word2vec + CNN delivered the strongest overall performance (AUPRC = 0.7550), combining high precision (0.9074) with strong recall (0.6901). The highest F1 score (0.7840) demonstrates an effective balance between detecting vulnerabilities and limiting false alarms. Harer's CNN + ET achieved a competitive ranking performance (AUPRC = 0.6690) with very high precision (0.9167) but substantially lower recall (0.4648), suggesting a conservative classifier that prioritizes prediction accuracy over vulnerability coverage. Russell's CNN exhibited balanced detection capability (AUPRC = 0.6553), achieving the highest recall (0.7465) among all models while maintaining good precision (0.6974). This produced one of the best precision–recall trade-offs (F1 = 0.7211). Lastly, Russell's CNN + RF combined strong ranking performance (AUPRC = 0.7208) with high precision (0.8545) and good recall (0.6620). Figure 1 shows the Precision-Recall (PR) curves for the reproduced models, and Figure 2 shows their AUPRC + CI Bar chart.

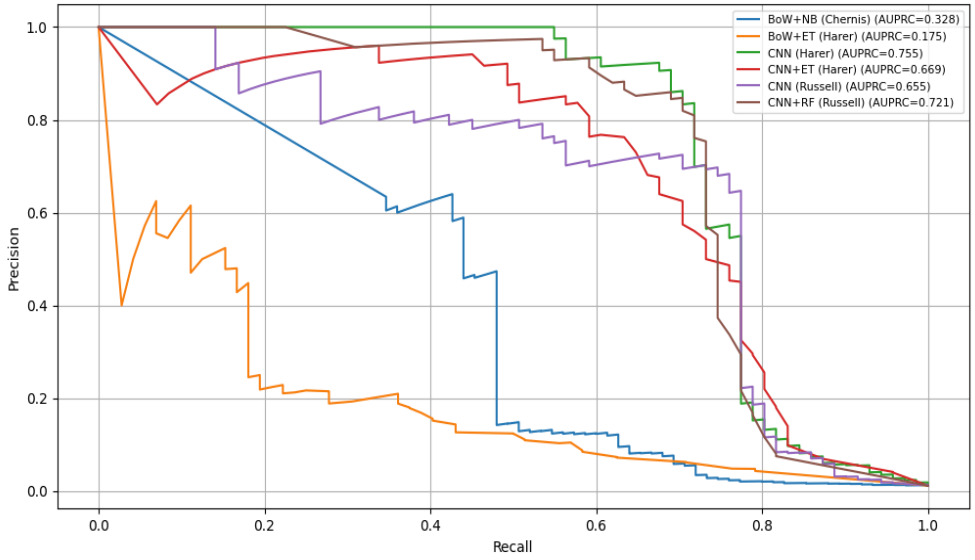


Figure 1: Precision-Recall Curves for all models

The hybrid architecture maintained a favorable balance between accurate positive predictions and vulnerability detection, yielding the second-highest AUPRC and F1 score. The precision–recall curves for all reproduced models in Figure 1 show that neural architectures consistently dominate the upper region of the precision–recall space, maintaining higher precision across a wide range of recall values compared to classical approaches. In contrast, BoW-based models exhibit rapid degradation in precision, indicating poor ranking performance under class imbalance. Among neural models, CNN-based architectures show similar

performance profiles, with overlapping and intersecting curves, suggesting comparable effectiveness across operating thresholds. Hybrid models demonstrate mixed behavior: CNN + RF closely tracks the best-performing models, while CNN + ET exhibits earlier degradation in precision.

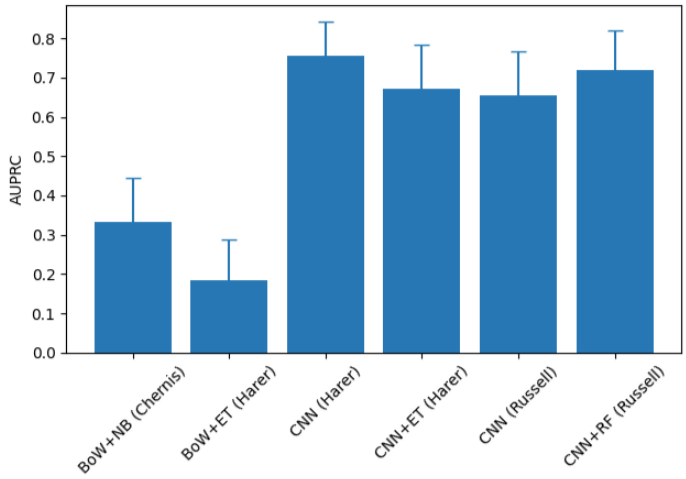


Figure 2: Model Performance with 95% Confidence Intervals

As also shown in Figure 2, neural and hybrid models substantially outperform classical BoW-based approaches, with clear separation in AUPRC values. While Harer's CNN achieves the highest mean performance, its confidence interval overlaps with that of Russell's CNN + RF, indicating comparable performance. Hybrid models exhibit greater variability, particularly CNN + ET, which shows wider confidence intervals.

Table 2 compares reproduced AUPRC values, which reflect the overall ranking performance of the individual models, with the AUPRC values reported in prior studies where available. Chernis and Verma [17] used Accuracy as an evaluation metric on a balanced dataset. This may not apply to our imbalanced-data case, but we reported it for transparency. For Russell's models, the study reported experimental results on two datasets, which we tagged Dataset A and B, for convenience. For Dataset A, the reproduced results preserve the relative ordering reported in the original work: CNN + RF outperforms standalone CNN in both cases. Specifically, the reproduced AUPRC values of 0.7208 (CNN + RF) and 0.6553 (CNN) align with the reported AUPRC trend (0.518 vs 0.467), indicating consistent model behavior despite differences in absolute values. For Harer's models, larger deviations are observed. The reproduced CNN achieves an AUPRC of 0.7550, substantially higher than the reported AUPRC (0.45), while the BoW + ET model drops from a reported 0.44 to 0.1748. These discrepancies suggest sensitivity to dataset characteristics. Nevertheless, the reproduced results maintain the expected hierarchy in which neural models outperform classical approaches.

Table 2: Comparison of Reported Metrics in Prior Studies and Reproduced AUPRC values

Model	Reported Metric	Reported value	Reproduced AUPRC
Chernis BoW + NB	Accuracy	0.635	0.3279
Harer's BoW + NB	AUPRC	0.44	0.1748
Harer's word2vec + CNN	AUPRC	0.45	0.7550
Harer's CNN + ET	AUPRC	0.49	0.6690
Russell's CNN	AUPRC (Dataset A)	0.467	0.6553
	AUPRC (Dataset B)	0.944	
Russell's CNN + RF	AUPRC (Dataset A)	0.518	0.7208
	AUPRC (Dataset B)	0.916	

We conducted a stability analysis of the 95% CI results to quantify how reliable and consistent the model's performance is under data variation. The mean bootstrap AUPRC is the average AUPRC across the 100 bootstrap resamples of the test set. CI widths are calculated from the 95% CI (low, high) values presented in Table 1. Smaller confidence interval widths indicate greater precision of the estimated AUPRC and, therefore, greater empirical stability of model performance under bootstrap resampling. Table 3 presents the stability analysis results.

Table 3: Stability Analysis of Models Based on Bootstrap Confidence Intervals Widths

Model	Mean Bootstrap AUPRC	CI Width
Chernis BoW + NB	0.3319	0.2188
Harer's BoW + NB	0.1846	0.1848
Harer's word2vec + CNN	0.7546	0.1822
Harer's CNN + ET	0.6709	0.2427
Russell's CNN	0.6553	0.2278
Russell's CNN + RF	0.7198	0.1971

In this study, model stability is quantified by the width of the bootstrap confidence interval around the estimated mean bootstrap AUPRC. Narrower confidence intervals indicate greater precision in the performance estimate and, therefore, greater empirical stability under repeated sampling. We interpret stability comparatively across reproduced models rather than using predefined categorical thresholds. Harer's word2vec + CNN exhibited the narrowest confidence interval (width = 0.1822), indicating the most stable performance estimate among the evaluated models, whereas Harer's CNN + ET (width = 0.2478) produced the widest interval, reflecting greater uncertainty in its estimated ranking performance.

In contrast, hybrid models, particularly CNN + ET, exhibit increased instability despite competitive performance. This suggests that hybridization may introduce additional variance, highlighting a trade-off between performance and robustness. The comparative evaluation of the reproduced models, presented in Table 3, reveals a clear performance hierarchy when ranked by mean bootstrap AUPRC. Harer's CNN achieves the highest mean AUPRC of 0.7546, followed by Russell's CNN + RF (0.7198), Harer's CNN + ET (0.6709), and Russell's CNN (0.6553). Classical models perform substantially worse, with Chernis' BoW + NB

achieving 0.3319, and Harer's BoW + ET yielding the lowest performance at 0.1846. Table 4 presents the model ranking based on mean bootstrap AUPRC.

Table 4: Model Ranking Based on Mean Bootstrap AUPRC Performance

Rank	Model	Mean Bootstrap AUPRC
1	Harer's word2vec + CNN	0.7546
2	Russell's CNN + RF	0.7198
3	Harer's CNN + ET	0.6709
4	Russell's CNN	0.6553
5	Chernis' BoW + NB	0.3319
6	Harer's BoW + ET	0.1846

Furthermore, to inferentially assess the reproduced models' performance, we conducted a pairwise statistical analysis using stratified bootstrap. Neural models significantly outperform classical approaches, as evidenced by large effect sizes and confidence intervals that do not cross zero. Hybridization improves performance in Russell's architecture, where CNN + RF significantly outperforms the standalone CNN. However, the top-performing neural models show no statistically significant differences, reinforcing their comparable effectiveness. Table 5 presents the pairwise statistical comparison using stratified bootstraps of ΔAUPRC, 95% CI, and Significance at $\alpha = 0.05$.

Table 5: Pairwise Statistical Comparison of Models Using Stratified Bootstrap (ΔAUPRC, 95% CI, and Significance at $\alpha = 0.05$)

Model A	Model B	ΔAUPRC	95% CI	p-value	Significant (α=0.05)
Harer's word2vec + CNN	Harer's CNN + ET	0.0849	[-0.0116, 0.1828]	0.10	No
Russell's CNN	Russell's CNN + RF	-0.0646	[-0.1271, -0.0065]	0.024	Yes
Harer's word2vec + CNN	Russell's CNN + RF	0.0347	[-0.0342, 0.1156]	0.31	No
Harer's word2vec + CNN	Harer's BoW + ET	0.7420	[0.6452, 0.8271]	0.001	Yes
Harer's BoW + ET	Chernis' BoW + NB	0.1713	[0.0987, 0.2583]	0.001	Yes
Harer's word2vec + CNN	Harer's CNN + ET	0.0849	[-0.0116, 0.1828]	0.10	No

5 DISCUSSION OF FINDINGS

5.1 Overall Performance Under Shared Evaluation

This is investigated by RQ1: How do representative token-based vulnerability detection systems perform when faithfully reproduced and evaluated under a shared dataset and identical data splits?

The results indicate that reproduction fidelity is partial and is better reflected in relative model behavior than in absolute metric agreement. Across models, reproduced AUPRC overall ranking performance values (presented in Table 1) differ substantially from those reported in prior studies. For example, Harer's word2vec CNN achieves an AUPRC of 0.7550 compared to the reported AUPRC of 0.45, while Harer's BoW + ET drops from 0.44 to 0.1748. Similarly, Russell's models show higher reproduced AUPRC values (0.6553 and 0.7208) relative to their reported AUPRC on Dataset A (0.467 and 0.518), while remaining far below the results reported on their Dataset B (≈0.91–0.94).

Despite these discrepancies, relative performance trends are largely preserved. In both reproduced and reported results, neural models outperform classical approaches, and hybrid CNN + RF outperforms standalone CNN in Russell's setting. This consistency suggests that model ranking is more reproducible than absolute performance values. The observed deviations likely reflect differences in dataset characteristics, class imbalance, and preprocessing pipelines. This is further supported by the large variation in AUPRC across datasets in Russell's study, reinforcing that dataset dependence is a primary driver of performance variability.

The challenges observed in cross-study vulnerability detection evaluation are consistent with broader concerns about empirical comparability in software security research. In their survey of automated fuzz-driver generation, Feng et al. [20] identify inconsistencies in benchmark selection, measurement criteria, validation procedures, and cost reporting as barriers to meaningful cross-study comparison. They argue that reporting results alone is insufficient when methodological differences prevent researchers from determining whether observed improvements are directly comparable. This study addresses this concern by reproducing representative token-based detection systems on a common dataset with identical train, validation, and test partitions, while preserving model-specific preprocessing and architectural configurations.

Answer to RQ1: Reproduction fidelity is limited in absolute terms but consistent in relative model behavior, indicating that prior findings are reproducible at the level of performance trends rather than exact metrics.

5.2 Comparative Performance Across Architectural Paradigms

This is investigated by RQ2: Under identical dataset partitions, how do classical count-based models compare to deep and hybrid neural architectures?

Neural architectures consistently outperform classical models, confirming that learned representations capture richer semantic and structural information than token-frequency features. This is supported by both the magnitude of the performance and its statistical significance. Model comparison based on mean AUPRC establishes a clear and statistically grounded performance hierarchy. Harer's word2vec CNN ranks highest with a mean AUPRC of 0.7546, followed by Russell's CNN + RF (0.7198), Harer's CNN + ET (0.6709), and Russell's CNN (0.6553). Classical models perform substantially worse, with Chernis' BoW + NB at 0.3319 and Harer's BoW + ET at 0.1846. These results define three distinct performance tiers:

- Top tier: Harer CNN, Russell CNN + RF

- Middle tier: Hybrid CNN + ET and standalone CNN
- Bottom tier: BoW-based classical models

Pairwise statistical testing provides further insight into these differences. The improvement of Russell's CNN + RF over its standalone CNN is statistically significant (ΔAUPRC = -0.0646, $p = 0.024$), confirming the benefit of hybridization in this setting. In contrast, the difference between Harer's CNN and CNN + ET is not statistically significant ($p = 0.10$), despite a higher mean bootstrap AUPRC for the standalone CNN.

Similarly, the difference between the two top-performing models, Harer's CNN and Russell's CNN + RF, is not statistically significant ($p = 0.31$), indicating comparable performance. However, neural models significantly outperform classical approaches, as evidenced by large effect sizes and highly significant differences (e.g., CNN vs BoW + ET, $p < 0.001$).

Answer to RQ2: Neural models significantly outperform classical approaches, while hybrid models provide context-dependent improvements. Differences among top-performing neural models are not statistically significant, indicating comparable effectiveness at the upper performance tier.

5.3 Impact of Hybridization

This is investigated by RQ3: Does combining neural feature extraction with ensemble classifiers consistently improve performance compared to standalone neural models?

The results reveal several important architectural insights regarding representation learning and hybridization. The effectiveness of hybrid models is inconsistent and cannot be generalized across architectures. In Russell's case, CNN + RF significantly outperforms the standalone CNN, suggesting the learned feature space suits tree-based classification. In contrast, Harer's CNN + ET does not significantly outperform the standalone CNN and exhibits lower mean performance, indicating that hybridization can degrade performance when feature–classifier compatibility is weak.

The stability analysis further shows that hybrid models may introduce additional variability. Harer's CNN + ET has the widest confidence interval (CI width = 0.2427), indicating low stability, while standalone CNN models strike a better balance between performance and uncertainty. Finally, threshold analysis reveals substantial variation across models (e.g., 0.17 vs 0.77), suggesting differences in probability calibration and decision boundaries. This further indicates that architectural choices may influence not only ranking performance but also predictive confidence behavior.

Answer to RQ3: Neural models provide the strongest and most consistent performance, while hybrid models offer conditional benefits depending on feature–classifier alignment and may introduce additional variability. Architectural design may therefore play a critical role in both performance and stability.

5.4 Summary of Findings

First, neural models consistently outperform classical approaches, confirming the importance of learned representations over token-frequency features. This performance gap is substantial and stable across evaluations. Second, hybrid models exhibit mixed effectiveness. In Harer's setting, CNN + ET underperforms the standalone CNN, whereas in Russell's setting, CNN + RF improves upon the standalone model. This suggests that the benefit of hybridization depends on the compatibility between learned feature representations and the downstream classifier. Third, confidence interval analysis reveals variability across architectures. Hybrid models, particularly CNN + ET, exhibit the widest confidence intervals (e.g., CI width

0.2427), indicating lower stability. In contrast, standalone neural models provide a more consistent balance between performance and uncertainty.

6 THREATS TO VALIDITY

The dataset used is a confirmed ground-truth dataset for function-level vulnerabilities, and the use of precision-recall metrics addresses the class imbalance in the dataset. However, faithfully reproducing the original studies' architectural characteristics and preserving the original hyperparameters may be suboptimal for the shared dataset, especially since no cross-model normalization was performed. These concerns are addressed by using shared dataset partitions across all methods, fixed random seeds, and statistical significance testing via bootstrap. External validity concerns generalizability beyond the evaluated dataset, especially since the evaluation is limited to C/C++ code and therefore may not generalize to other programming languages. Threshold tuning may not reflect deployment scenarios. This study focuses on comparative generalization under shared data, not universal deployment performance. Conclusion validity concerns statistical inference reliability. Examples of this are (a) an increase in Type I error in multiple comparisons, (b) bootstrapped confidence intervals' dependence on test set size, and (c) statistically significant AUPRC differences even when practically small. We reduced this by employing pairwise bootstrap tests and reporting effect sizes alongside p-values.

7 CONCLUSION

This study shows that reproduction fidelity in software vulnerability detection is better assessed by relative model behavior than by absolute metric agreement. We observed substantial discrepancies between reproduced and reported AUPRC values, driven by differences in dataset composition, class imbalance, and preprocessing pipelines. The preservation of performance ordering across models indicates that the core methodological insights of prior work remain valid. These findings suggest that conclusions drawn in earlier studies are reproducible at the level of comparative trends, even when exact performance values are not. The substantial differences between reproduced and originally reported performance values, alongside the preservation of several relative performance trends, illustrate why standardized evaluation conditions and transparent reporting are necessary for interpreting the generalizability of previously published results.

The comparative evaluation further establishes that neural architectures consistently outperform classical token-based approaches in both ranking performance and overall effectiveness. This reinforces the importance of learned representations in capturing the semantic and structural properties of source code. However, the results challenge the assumption that hybrid models universally improve performance. The contrasting behavior of CNN + ET and CNN + RF shows that hybridization works differently depending on the alignment between learned feature representations and the inductive bias of downstream classifiers. In practice, hybrid models may yield improvements, but they can also introduce performance degradation or instability.

Incorporating stability analysis provides additional insight into model reliability. Although neural models achieve higher AUPRC, they do not consistently exhibit lower uncertainty. Notably, hybrid models tend to display greater variability, indicating a potential trade-off between performance and robustness. This highlights the importance of reporting confidence intervals alongside point estimates, particularly in imbalanced learning settings where performance variability can be pronounced.

Overall, this study emphasizes the need for standardized evaluation protocols, consistent metrics, and statistically grounded comparisons. By demonstrating that model performance, stability, and relative behavior can vary significantly under different experimental conditions, we highlight the importance of controlled and reproducible evaluation frameworks for advancing the state of the art in software vulnerability detection.

ACKNOWLEDGMENTS

This work used resources of the Center for Computationally Assisted Science and Technology (CCAST) at North Dakota State University, which were made possible in part by National Science Foundation Major Research Instrumentation (MRI) Award No. 2019077.

REFERENCES

[1] P. E. Black, L. Badger, B. Guttman, and E. Fong, "Dramatically reducing software vulnerabilities: Report to the White House Office of Science and Technology Policy," National Institute of Standards and Technology, Gaithersburg, MD, NIST IR 8151, Nov. 2016. doi: 10.6028/NIST.IR.8151.

[2] S. Perera, X. Jin, A. Maurushat, and D.-G. J. Opoku, "Factors Affecting Reputational Damage to Organisations Due to Cyberattacks," *Informatics*, vol. 9, no. 1, p. 28, Mar. 2022, doi: 10.3390/informatics9010028.

[3] TechTarget, "How to Deliver DevSecOpsVeracode." VeraCode, 2018.

[4] Edgescan, "2016 Vulnerability Statistics Report." EdgeScan Continous Vulnerability Management, 2016. [Online]. Available: www.edgescan.com

[5] "Checkmarx – Application Security, Made Easy," Checkmarx. Accessed: Sep. 14, 2020. [Online]. Available: https://www.checkmarx.com/

[6] "Pysa: Open Source static analysis for Python code," Facebook Engineering. Accessed: Sep. 09, 2020. [Online]. Available: https://engineering.fb.com/security/pysa/

[7] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, Jul. 2022, doi: 10.1109/TDSC.2021.3051525.

[8] I. Medeiros, N. Neves, and M. Correia, "DEKANT: a static analysis tool that learns to detect web application vulnerabilities," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, Saarbrücken Germany: ACM, Jul. 2016, pp. 1–11. doi: 10.1145/2931037.2931041.

[9] H. K. Dam, T. Tran, T. Pham, S. W. Ng, J. Grundy, and A. Ghose, "Automatic Feature Learning for Predicting Vulnerable Software Components," *IEEE Trans. Softw. Eng.*, vol. 47, no. 1, pp. 67–85, Jan. 2021, doi: 10.1109/TSE.2018.2881961.

[10] H. Perl *et al.*, "VCCFinder: Finding Potential Vulnerabilities in Open-Source Projects to Assist Code Audits," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Denver Colorado USA: ACM, Oct. 2015, pp. 426–437. doi: 10.1145/2810103.2813604.

[11] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, "Predicting vulnerable software components," in *Proceedings of the 14th ACM conference on Computer and communications security*, Alexandria Virginia USA: ACM, Oct. 2007, pp. 529–540. doi: 10.1145/1315245.1315311.

[12] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," in *33rd Conference on Neural Information Processing Systems*, Vancouver, BC, Canada: NeurIPS, 2019, p. 11.

[13] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "μVulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection," *IEEE Trans. Dependable Secure Comput.*, pp. 1–1, 2019, doi: 10.1109/TDSC.2019.2942930.

[14] F. Wu, J. Wang, J. Liu, and W. Wang, "Vulnerability detection with deep learning," in *2017 3rd IEEE International Conference on Computer and Communications (ICCC)*, Chengdu: IEEE, Dec. 2017, pp. 1298–1302. doi: 10.1109/CompComm.2017.8322752.

[15] G. Lin *et al.*, "Cross-Project Transfer Representation Learning for Vulnerable Function Discovery," *IEEE Trans. Ind. Inform.*, vol. 14, no. 7, pp. 3289–3297, Jul. 2018, doi: 10.1109/TII.2018.2821768.

[16] J. A. Harer *et al.*, "Automated software vulnerability detection with machine learning," Aug. 02, 2018, *arXiv*: arXiv:1803.04497. doi: 10.48550/arXiv.1803.04497.

[17] B. Chernis and R. Verma, "Machine Learning Methods for Software Vulnerability Detection," in *Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics*, Tempe AZ USA: ACM, Mar. 2018, pp. 31–39. doi: 10.1145/3180445.3180453.

[18] R. L. Russell *et al.*, "Automated Vulnerability Detection in Source Code Using Deep Representation Learning," Nov. 28, 2018, *arXiv*: arXiv:1807.04320. doi: 10.48550/arXiv.1807.04320.

[19] H. Xie, Y. Shao, Z. Li, Z. Alomari, and A. Makanju, "Optimization of Class Imbalance Techniques in Machine Learning Models for Network Intrusion Detection," in *2025 9th International Conference on Cryptography, Security and Privacy (CSP)*, Okinawa, Japan: IEEE, Apr. 2025, pp. 102–106. doi: 10.1109/CSP66295.2025.00025.

[20] X. Feng *et al.*, "Fuzz Driver Generation: A Survey and Outlook from the Perspective of Data Sources," *Big Data Cogn. Comput.*, vol. 10, no. 4, p. 129, Apr. 2026, doi: 10.3390/bdcc10040129.

Authors' background

Your Name	Title*	Research Field	Personal website
Semiu Akanmu	PhD Candidate	Machine Learning for Software Security	
Simone Ludwig	Professor	Machine Learning; Evolutionary Algorithm	

*This form helps us to understand your paper better, the form itself will not be published.

*Title can be chosen from: master student, Phd candidate, assistant professor, lecture, senior lecture, associate professor, full professor